CAMPUSCONNECT EVENTS

Project Report and Technical Documentation

College Event Registration System | *MERN Stack*

Submitted by: Akhil Mani

Register/Roll Number: 225455

Department: B tech CSE

Institution: sahrdaya college of engineering and technology thrissur

Academic Year: 2026 - 27

Project Guide: Dr.Anil Antony

1. Executive Summary

CampusConnect Events is a web-based college event registration system. It gives visitors a way to browse events, allows students to create accounts and manage event registrations, and provides administrators with event-management and registration-list capabilities. The supplied source code uses React with Vite for the frontend and Node.js, Express, Mongoose, and MongoDB for the backend.

This report describes the system based on the supplied repository files. Deployment URLs, operational test results, and institutional details should be added by the project owner where appropriate; this document does not claim that every test has been run in production.

2. Problem Statement

College events are often announced and managed through disconnected forms, messages, and spreadsheets. This can make it difficult for organizers to monitor registration counts, enforce capacity limits, and retrieve attendee details. CampusConnect Events centralizes event information and registration records behind a web interface and API.

3. Objectives

Provide a single place to browse, search, and view college events.

Support student sign-up, login, event registration, cancellation, and registration history.

Allow administrators to create, edit, and delete events and view attendee lists.

Enforce event capacity and prevent duplicate registrations.

Separate the browser interface, API logic, and persistent data storage.

Use server-side validation and role-based access checks for protected operations.

4. Scope and User Roles

Role	Capabilities described by the project
Visitor	Browse and search event listings; view event details.
Student	Register and log in; register for eligible events; cancel registrations where allowed; view own registrations.
Admin / organiser	Create, update, and delete events; view per-event registration lists; export registration data as CSV where implemented.

Features identified as future enhancements in the project README include email confirmations, QR-code check-in, waiting lists, event images, password reset, and event reminders. These should not be represented as completed features unless they have since been implemented.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, JavaScript, CSS	Single-page web interface and client-side routing/state.
Backend	Node.js, Express	HTTP API, route handling, middleware, authentication and business rules.
Database	MongoDB with Mongoose	Persistence for users, events, and registrations.
Authentication	JWT and bcrypt (per project documentation)	Token-based authentication and password hashing.
Validation / middleware	Express validation, custom middleware, Helmet, express-rate-limit, CORS	Input validation, security headers, request limits, and browser-origin controls.
Hosting	Render (deployed by project owner)	Backend and frontend hosting; exact service URLs should be recorded by the owner.

6. System Architecture

The browser-based React client sends HTTP requests to the Express API, which applies validation and authentication/authorization middleware before controllers interact with Mongoose models. MongoDB stores the persistent records. The client should never connect directly to MongoDB.

Component	Responsibilities
React client	Pages, reusable UI components, auth context, fetch/loading/error handling, API client.
Express API	Route mapping, request validation, controllers, centralized error handling, CORS policy, rate limiting.
Mongoose models	Schema validation, indexes, references between users, events, and registrations.
MongoDB	Persistent application data.

7. Main Functional Modules

7.1 Authentication and Accounts

The API exposes registration, login, and current-user routes. User records include name, email, password hash, role, department, and roll number. The model excludes the password hash from normal query results/JSON serialization. The documented roles are student and admin.

7.2 Event Catalogue

Public event listing supports pagination and optional title search, category filtering, and upcoming-event filtering. Event details include title, description, category, venue, event date, registration deadline, capacity, and registered count. Derived values include seats remaining and registration status.

7.3 Registration Management

Students can register for events while registration is open and seats remain. The registration model has a unique compound index on user and event, which prevents the same user from registering for the same event more than once. The registration controller uses an atomic event-count update when reserving a seat and attempts to restore the count if saving the registration fails.

7.4 Administration

Admin-protected endpoints create, update, and delete events and retrieve the list of attendees for an event. The event controller restricts changes to explicitly editable fields and prevents event capacity from being lowered below the current registered count. The README documents CSV export as a user-facing capability; verify the current UI and route implementation before claiming it in a formal demonstration.

7.5 Validation and Error Handling

The backend includes validation middleware, validators, an asynchronous controller wrapper, an API error utility, and centralized error handling. Helmet, rate limiting, and CORS are configured in the Express application.

8. Data Model

Collection / Model	Key fields	Constraints / notes
User	name, email, passwordHash, role, department, rollNumber, timestamps	Email is unique and lowercased; role is student/admin; passwordHash is excluded from normal selection.
Event	title, description, category, venue, date, registrationDeadline, capacity, registeredCount, createdBy, timestamps	Category enum; positive integer capacity; date/category indexes; derived seatsLeft and registrationStatus.
Registration	user reference, event reference, timestamps	Unique compound index on (user, event); indexed by event and creation time.

9. API Overview

Method	Endpoint	Access / purpose
GET	/api/health	Public health check.
POST	/api/auth/register	Public; create student account.
POST	/api/auth/login	Public; authenticate user.
GET	/api/auth/me	Authenticated; retrieve current user.
GET	/api/events	Public; list/search/filter/paginate events.
GET	/api/events/:id	Public; retrieve event details.
POST	/api/events	Admin; create event.

PUT	/api/events/:id	Admin; update event.
DELETE	/api/events/:id	Admin; delete event and related registrations.
POST	/api/events/:id/register	Student; register for event.
DELETE	/api/events/:id/register	Student; cancel registration where permitted.
GET	/api/registrations/me	Student; retrieve own registrations.
GET	/api/events/:id/registrations	Admin; retrieve attendee list.

All endpoints are under the /api prefix. The exact request fields and response formats should be verified against the current route/controller code before publishing a formal API contract.

10. Security Considerations

Store MongoDB credentials, JWT secrets, and admin credentials only in environment variables; never commit real .env files.

Rotate any database password or token secret that has been exposed in chat, screenshots, logs, or a public repository.

Use a strong, unique JWT secret in production.

Use a least-privilege MongoDB database user and restrict network access to the extent supported by the hosting setup.

Set CLIENT_ORIGIN to the exact deployed frontend origin and verify that the CORS configuration parses comma-separated origins correctly.

Use HTTPS for public deployments and avoid logging passwords, tokens, or connection strings.

Review the deployed API and authentication behavior before using real student information.

11. Installation and Local Execution

Install Node.js and npm, Git, and either a local MongoDB server or a MongoDB Atlas database.

Clone the GitHub repository or extract the project ZIP, then open the project root in VS Code.

In a terminal, enter the server folder and run: npm install.

Copy server/.env.example to server/.env and set MONGODB_URI, a strong JWT_SECRET, JWT_EXPIRES_IN, CLIENT_ORIGIN, and seed admin values as required by the code.

From the server folder, run npm run seed if you intend to create the initial admin account and sample events; confirm the seed script behavior before running against a production database.

Start the backend using the script actually defined in server/package.json (commonly npm run dev locally).

In a second terminal, enter client and run npm install.

Copy client/.env.example to client/.env and set VITE_API_URL to the local API base URL, commonly http://localhost:5000/api.

From client, run npm run dev and open the URL printed by Vite, commonly http://localhost:5173.

Environment variable names and start scripts should be checked against the current .env.example and package.json files. Do not assume the example URLs are the production URLs.

12. Deployment Notes

Provision a MongoDB Atlas database and use its connection string as MONGODB_URI in the backend host's environment settings.

Deploy the server folder as a Node Web Service. Configure its root directory as server, build command as npm install, and start command according to server/package.json.

Deploy the client folder as a Static Site. Configure root directory client, build command npm install && npm run build, and publish directory dist.

Set VITE_API_URL in the frontend build environment to the backend public URL followed by /api, then trigger a new frontend build.

Set CLIENT_ORIGIN on the backend to the exact frontend public origin (no /api suffix); redeploy/restart the backend after changes.

Verify /api/health, event listing, registration, login, and admin-only routes. Review Render logs and browser console if a request fails.

Deployment providers can change their free-tier policies and dashboard labels. Confirm current platform requirements in the provider dashboard.

13. Testing Plan

Test case	Expected result	Actual result
Health endpoint	GET /api/health returns status ok.	To be recorded
Valid student registration	Account created and authentication response returned.	To be recorded
Duplicate email	Request rejected with a clear error.	To be recorded
Invalid login	Authentication rejected without revealing sensitive details.	To be recorded
Public event list	Events returned with pagination metadata.	To be recorded
Unauthenticated admin action	Request rejected.	To be recorded
Student attempts admin route	Request forbidden.	To be recorded
Duplicate event registration	Second registration rejected.	To be recorded
Event capacity reached	Additional registration rejected.	To be recorded
Cancel registration	Registration removed and seat count adjusted.	To be recorded

CORS production origin	Browser request from configured frontend origin succeeds.	To be recorded
Responsive UI	Core screens remain usable on narrow screens.	To be recorded

These are proposed verification cases based on the project design, not a claim that all tests have already passed.

14. Limitations and Future Enhancements

Email confirmation and reminder workflows.

QR-code attendance/check-in.

Waiting list for full events.

Event images and richer media support.

Password reset and account recovery.

Automated test suite and CI checks.

Accessibility audit and usability testing with actual students.

Operational monitoring, backups, and documented recovery procedures.

15. Conclusion

CampusConnect Events provides a modular starting point for centralizing college event discovery and registration. Its MERN architecture separates the user interface, API, and database, while the data models and controllers implement core registration constraints. Before formal submission or use with real student data, the project should be tested end to end, deployment settings and secrets should be reviewed, and actual test outcomes should be recorded.

Appendix A: Repository Structure

```
event-registration/
  client/
    src/
      components/
      context/
      hooks/
      pages/
      services/
      utils/
    public/
    package.json
    vite.config.js
  server/
    src/
```

```
    config/
    controllers/
    middleware/
    models/
    routes/
    utils/
    validators/
    app.js
    server.js
    seed.js
  package.json
  .env.example
README.md
.gitignoreevent-registration/
client/
  src/
    components/
    context/
    hooks/
    pages/
    services/
    utils/
  public/
  package.json
  vite.config.js
server/
  src/
    config/
    controllers/
    middleware/
    models/
    routes/
    utils/
    validators/
    app.js
    server.js
    seed.js
  package.json
  .env.example
README.md
.gitignore
```